

СОДЕРЖАНИЕ

Введение.....	6
1 Обзор литературы	7
1.1 Нейронная сеть	7
1.2 Google MediaPipe.....	8
1.3 Telegram Bot API	10
2 Системное проектирование.....	12
2.1 Перечень структурных блоков проекта	12
2.2 Блок взаимодействия с пользователем	12
2.3 Блок интеграции с телеграм-ботом	13
2.4 Блок модели нейронной сети	13
2.5 Блок обучения нейронной сети.....	13
2.6 Блок тестирования.....	14
2.7 Блок подсчета потерь.....	15
2.8 Блок набора данных	15
2.9 Блок подготовки набора данных	16
3 Функциональное проектирование	17
3.1 Архитектура программного модуля.....	17
3.2 Блок модели нейронной сети	17
3.1.1 Энкодер.....	17
3.1.2 Декодер	18
3.3 Блок подготовки набора данных	20
3.4 Блок обучения нейронной сети.....	21
3.5 Блок интеграции с Telegram-ботом	21
3.6 Зависимости и организация проекта	22
7 Техничко-экономическое обоснование разработки и реализации на рынке телеграм-бота с поддержкой языка глухонемых	23
7.1 Характеристика программного средства, разрабатываемого для реализации на рынке.....	23
7.2 Расчет инвестиций в разработку программного средства для его реализации на рынке.....	23
7.3 Расчет экономического эффекта от реализации программного.....	25
7.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке	27
7.5 Анализ полученных результатов	27
Заключение	29
Список использованных источников	30
Приложение А	31
Приложение Б.....	32
Приложение В.....	33

ВВЕДЕНИЕ

В настоящее время мессенджер Telegram активно используется широким кругом пользователей для обмена информацией, и голосовые сообщения становятся все более популярным средством коммуникации. Многие пользователи предпочитают голосовое общение за его оперативность и эмоциональную окраску. Однако для людей с нарушениями слуха и речи традиционные аудиосообщения недоступны, что создает серьезный информационный барьер. Современные достижения в области обработки изображений и видеопотока, а также глубокого обучения, открывают новые возможности для разработки систем, способных адаптировать коммуникационные сервисы под нужды данной категории пользователей.

За последнее десятилетие нейронные сети зарекомендовали себя как эффективный инструмент для обработки и анализа больших объемов данных, в том числе и изображений. Их способность автоматически извлекать сложные признаки из визуальной информации существенно повышает точность распознавания объектов, что критически важно для задач компьютерного зрения. Современные архитектуры позволяют нейросетевым моделям адаптироваться к разнообразию входных данных, обеспечивая надежное распознавание даже в условиях плохой освещенности.

Целью данного дипломного проекта является разработка Telegram-бота с поддержкой языка глухонемых, который обеспечит доступность коммуникации для людей с нарушениями слуха и речи путем анализа видеопотока и трансляции его в текст. Проект предназначен для людей с нарушением речи и слуха и разрабатывается для уравнивания возможностей между различными социальными группами.

В соответствии с поставленной целью можно выделить следующий набор задач данного дипломного проекта:

- выбор нейросетевой модели для реализации алгоритма распознавания жестов в видеопотоке;
- подготовка датасета для обучения модели;
- обучение нейросетевой модели на основе созданного датасета;
- разработка телеграм-бота для взаимодействия с моделью;
- интеграция модели с приложением мессенджера Telegram;
- проверка качества работоспособности и распознавания жестов в видеопотоке.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет XX% (отчет о проверке на заимствования прилагается).

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Нейронная сеть

Нейронные сети представляют собой вычислительные модели, созданные по аналогии с биологическими нейронными системами, лежащими в основе работы мозга живых существ. Они играют ключевую роль в современных подходах к машинному и глубокому обучению [1], обеспечивая возможность решения разнообразных сложных задач, включая классификацию, регрессию, генерацию данных, распознавание образов и обработку естественного языка. Благодаря высокой гибкости и способности адаптироваться к различным типам информации, нейронные сети широко используются в компьютерном зрении, анализе временных рядов, медицине и системах автономного управления.

Нейронные сети состоят из множества нейронов – взаимосвязанных узлов, организованных в структурированные слои. Каждый слой выполняет свою роль в обработке входной информации, преобразуя её шаг за шагом до получения окончательного результата. Важным аспектом является возможность обучения сети: она способна корректировать свои внутренние параметры на основе ошибок, выявленных в ходе обучения, что позволяет ей адаптироваться к новым данным и обнаруживать скрытые закономерности.

Современные нейронные сети строятся следующим образом:

1. Нейрон. Является основой нейронной сети и представляет собой базовую единицу сети, аналогичную клетке нервной системы. Каждый нейрон принимает входные сигналы, осуществляет над ними серию линейных преобразований с использованием весовых коэффициентов, а затем применяет нелинейную функцию активации. Эта функция позволяет ввести элемент нелинейности, благодаря чему сеть способна моделировать сложные зависимости. В процессе обучения веса нейронов корректируются, что повышает точность модели в решении поставленной задачи.

2. Слои нейронной сети. Нейроны в сети связываются в слои – структурное разделение, позволяющее организовать процесс обработки информации в несколько этапов:

- входной слой: принимает исходные данные, такие как пиксели изображения, звуковые сигналы или текстовые последовательности;
- скрытые слои: осуществляют обработку информации, глубина сети, то есть количество скрытых слоев и нейронов в каждом из них, определяет вычислительную сложность модели и её способность выявлять высокоуровневые абстракции;
- выходной слой: формирует конечный результат работы сети в зависимости от решаемой задачи.

3. Функции активации. Функция активации – это математическое преобразование, вводящее нелинейность в модель. Эти функции позволяют сети гибко адаптироваться к разнообразным входным данным и моделировать сложные зависимости.

4. Алгоритмы обучения. Алгоритм обучения – это процесс настройки весов нейронной сети для минимизации ошибки предсказания. Основным методом является алгоритм обратного распространения ошибки, в ходе которого вычисляется градиент функции потерь, а затем происходит корректировка весовых коэффициентов с использованием оптимизационных методов. Эти методы позволяют модели постепенно улучшать свою способность к обучению на новых данных.

Существует множество архитектур нейронных сетей, каждая из которых оптимизирована для решения определённого класса задач, ниже приведены наиболее распространённые из них:

1. Полносвязные нейронные сети (от англ. Fully Connected Neural Networks, FNN). В таких моделях каждый нейрон одного слоя соединён с каждым нейроном следующего слоя. Такие сети подходят для задач с небольшим числом признаков, однако при работе с изображениями или последовательностями они могут сталкиваться с проблемами масштабируемости и вычислительной затратности.

2. Сверточные нейронные сети (от англ. Convolutional Neural Networks, CNN). Эти сети особенно эффективны при обработке изображений и видео. Сверточные слои автоматически извлекают признаки из данных, сохраняя пространственную иерархию, что позволяет эффективно решать задачи классификации, сегментации и детектирования объектов.

3. Рекуррентные нейронные сети (от англ. Recurrent Neural Networks, RNN). Используются для работы с последовательными данными – временными рядами, текстом или аудиосигналами. Благодаря внутренней памяти, учитывающей предыдущие состояния, RNN способны моделировать контекст и зависимости, что делает их незаменимыми для задач перевода, синтеза речи и анализа последовательностей.

4. Генеративно-состязательные сети (от англ. Generative Adversarial Networks, GAN). Эти сети представляют собой две конкурирующие модели: генератор, создающий искусственные данные, и дискриминатор, оценивающий их достоверность. Такой состязательный процесс позволяет генерировать высококачественные изображения, видео и другие типы данных, приближенные к реальным.

Благодаря разнообразию архитектур и методов обучения, нейронные сети продолжают совершенствоваться, находя новые области применения. Их развитие способствует созданию всё более точных, эффективных и адаптивных моделей, способных решать широкий спектр задач в различных сферах науки и технологий – именно поэтому в данном проекте было решено их задействовать.

1.2 Google MediaPipe

Google MediaPipe [3] – это кросс-платформенный инструмент для обработки мультимедийных данных с применением методов машинного обучения, который помогает создавать высокоэффективные решения для

анализа видео, изображений, аудио и других данных в реальном времени. Этот фреймворк предоставляет гибкую архитектуру для создания цепочек обработки данных, которые могут быть адаптированы под задачи, такие как распознавание лиц, анализ движений или корректировка позы. MediaPipe обеспечивает модульную организацию процесса, позволяя кастомизировать цепочки обработки данных и интегрировать готовые модели машинного обучения, при этом обеспечивая высокую производительность на различных устройствах – от мобильных телефонов до серверных систем.

На рисунке 1.1 [2] приведены примеры типичных задач, решаемых с помощью модуля MediaPipe.

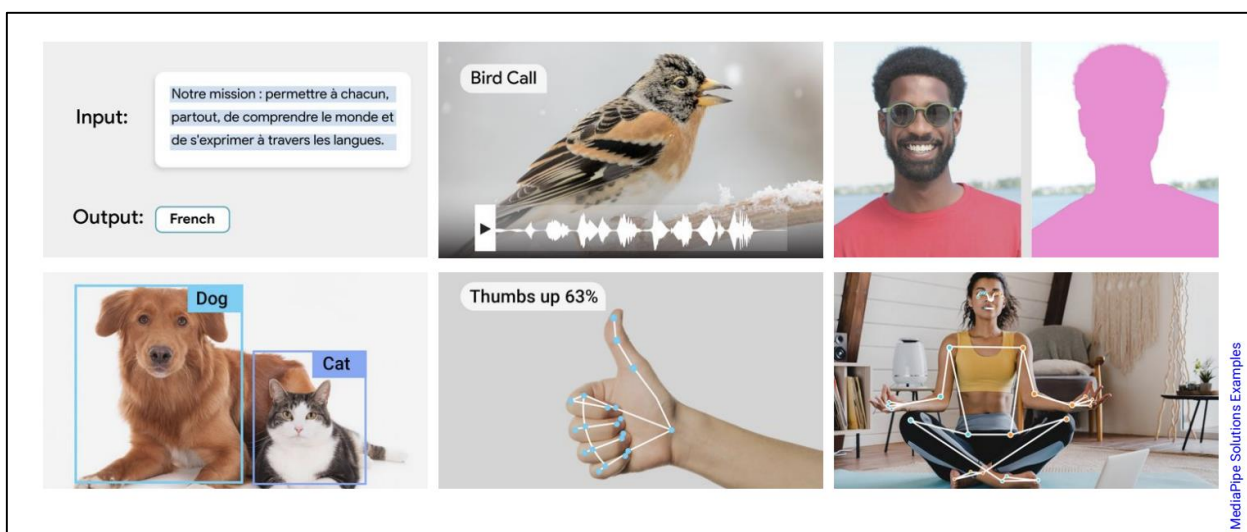


Рисунок 1.1 – Пример задач, решаемых с помощью MediaPipe

Основные элементы работы с Google MediaPipe:

1. Процесс обработки данных. Разработчику предоставляется возможность строить цепочки обработки данных, которые шаг за шагом обрабатывают и анализируют данные. Каждый этап в этих цепочках отвечает за выполнение определённой задачи, будь то простая фильтрация или же сложный алгоритм анализа изображения.

2. Использование готовых решений. Интегрирование готовых моделей машинного обучения, таких как детекторы лиц или сегменты для анализа изображений, значительно ускоряет разработку. В случае необходимости, системы можно дополнительно настроить, добавив специализированные алгоритмы.

3. Поддержка реального времени. MediaPipe оптимизирован для работы с потоковыми данными, что даёт возможность производить операции анализа и обработки данных в реальном времени без значительных задержек.

4. Многочисленные области применения. Благодаря универсальности, MediaPipe активно используется в разнообразных областях: от мобильных приложений и робототехники до систем видеонаблюдения и медицинских технологий. Фреймворк способствует быстрому созданию эффективных

решений для множества востребованных задач.

5. Интерфейсы и инструменты разработки. MediaPipe предоставляет удобные API и SDK для работы с такими языками программирования, как C++, Python и JavaScript. Это позволяет разработчикам создавать собственные решения, адаптировать существующие модули и интегрировать их в приложения на различных платформах, включая Android, iOS и веб.

1.3 Telegram Bot API

Telegram Bot API [4] – это интерфейс для разработки программных агентов или ботов в мессенджере Telegram, который позволяет автоматизировать взаимодействие с пользователями, обрабатывать входящие сообщения и отправлять ответы в режиме реального времени. Данный инструмент предназначен для упрощения создания сервисов, способных оперативно реагировать на запросы, выполнять различные задачи и интегрироваться с внешними системами, будь то информационные сервисы, системы поддержки клиентов или развлекательные проекты.

Основная идея Telegram Bot API заключается в использовании протокола HTTP для обмена данными между сервером Telegram и сервером разработчика. Разработчик получает обновления – новые сообщения, запросы или события – и обрабатывает их согласно заложенной логике приложения. Для получения обновлений предусмотрены два способа: периодический опрос сервера и автоматическая отправка обновлений на заранее настроенный адрес посредством веб-перехватчиков. Такой подход позволяет организовать надёжное и гибкое взаимодействие, отвечающее требованиям самых разнообразных проектов.

Среди возможностей API можно выделить широкий спектр функций:

- отправку текстовых сообщений, медиа-файлов, документов и местоположения;
- использование встроенных и настраиваемых клавиатур для упрощения взаимодействия с пользователем;
- поддержку групповых чатов и каналов;
- реализация сложных сценариев взаимодействия, таких как обработка запросов при вводе данных непосредственно в поле ввода.

Кроме того, API позволяет работать с анимациями, опросами и даже организовывать платёжные операции, что делает его универсальным инструментом для создания интерактивных и функциональных приложений.

Одним из значимых преимуществ Telegram Bot API является его простота интеграции с языком Python. Благодаря множеству специализированных библиотек, разработка ботов становится интуитивно понятной и быстрой. Эти библиотеки оборачивают все сложности взаимодействия с API, предоставляя высокоуровневые функции для отправки сообщений, обработки команд и работы с различными типами данных. Поддержка асинхронного программирования позволяет создавать масштабируемые решения, способные обрабатывать большое количество

одновременных запросов без потери производительности.

Работа с Telegram Bot API на Python часто строится вокруг двух основных подходов: постоянного опроса сервера и использования веб-перехватчиков. При первом подходе приложение регулярно запрашивает у серверов Telegram новые обновления, что удобно для разработки и тестирования, а также для небольших проектов. Второй метод, основанный на автоматической доставке обновлений на сервер разработчика, позволяет значительно снизить задержки и эффективно использовать ресурсы при создании коммерческих и высоконагруженных систем.

Кроме того, Python-среда предоставляет богатые возможности для отладки, тестирования и масштабирования ботов. Широкая экосистема инструментов и активное сообщество разработчиков способствуют быстрому решению возникающих задач, обмену опытом и постоянному обновлению библиотек в соответствии с изменениями Telegram Bot API. Это позволяет создавать надёжные и безопасные сервисы, отвечающие самым высоким стандартам качества и оперативности.

Telegram Bot API является мощным и универсальным инструментом для разработки ботов, способных автоматизировать взаимодействие с пользователями и решать широкий спектр задач. Его тесная интеграция с Python делает процесс создания и поддержки таких систем доступным даже для небольших команд, позволяя быстро внедрять инновационные решения в самые разные сферы деятельности.

2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

2.1 Перечень структурных блоков проекта

В данном дипломном проекте были выделены следующие структурные блоки:

1. Блок взаимодействия с пользователем.
2. Блок интеграции с телеграм-ботом.
3. Блок модели нейронной сети.
4. Блок обучения нейронной сети.
5. Блок тестирования.
6. Блок подсчета потерь.
7. Блок набора данных.
8. Блок подготовки набора данных.

Блоки выделены по принципу исполняемой задачи, то есть каждый блок представляет собой самостоятельный модуль проекта, выполняющий определенную функцию независимо от других блоков.

Подробное описание каждого блока, а также их взаимодействие между собой, представлено ниже. Схема структурная, включающая все перечисленные блоки и взаимодействие между ними, представлена на чертеже ГУИР.400201.074 С1.

2.2 Блок взаимодействия с пользователем

Блок взаимодействия с пользователем описывает взаимодействие конечного пользователя с системой. Взаимодействие осуществляется через интерфейс телеграм-бота, который принимает входные данные в виде видеозаписи или видео-сообщения. Для начала работы пользователю необходимо отправить боту соответствующий медиа-контент, содержащий жестовую речь. Бот поддерживает загрузку видеофайлов в различных форматах, а также видео-сообщений, записанных непосредственно в Telegram.

После получения видео бот обрабатывает его и направляет пользователю ответное текстовое сообщение, содержащее расшифровку переданных жестов. Коммуникация осуществляется в асинхронном режиме: пользователь отправляет видео, после обработки получает результат в виде текстового ответа. В случае возникновения ошибок, например, если формат видео не поддерживается или качество записи недостаточно для корректного распознавания, бот уведомляет пользователя об этом и предлагает повторить попытку с другими параметрами.

Кроме того, бот может предоставлять вспомогательную информацию, например, инструкции по использованию или рекомендации по качеству загружаемого видео. Это обеспечивает удобное и интуитивно понятное взаимодействие, позволяя пользователям получать текстовую интерпретацию жестовой речи без необходимости использования дополнительных инструментов.

2.3 Блок интеграции с телеграм-ботом

Блок обработки запросов описывает обработку входящих запросов в телеграм-боте в несколько этапов. После получения сообщения от пользователя бот выполняет декомпозицию запроса, определяя его тип и корректность данных. Если сообщение содержит видеофайл или видео-сообщение, система извлекает необходимые метаданные, проверяет формат и размер файла, а затем сохраняет его для дальнейшей обработки.

После успешного приема видео, формируется запрос к блоку обученной модели, передавая видеозапись для анализа.

В зависимости от успешности обработки, данный модуль отправляет в блок взаимодействия с пользователем расшифровку видеофайла, либо сообщение об ошибке.

2.4 Блок модели нейронной сети

Блок модели нейронной сети. Этот блок является ключевым компонентом системы, отвечающим за распознавание жестовой речи на видеозаписях и ее преобразование в текстовый формат. Входными данными для блока служит видеопоследовательность, содержащая язык жестов.

Основная задача модели – анализ движений рук, положения пальцев и мимики говорящего, а также интерпретация этих параметров в соответствии с заложенными языковыми правилами.

Процесс обработки видео основан на использовании модели MediaPipe, которая применяет методы компьютерного зрения для детекции и отслеживания ключевых точек руки. Сначала модель анализирует входное видео, выделяя последовательность кадров и извлекая координаты суставов и пальцевых сегментов. Затем эти данные передаются в классификационный модуль, который интерпретирует жесты на основе предварительно обученных шаблонов. Распознанные жесты сопоставляются с текстовыми эквивалентами, после чего модель формирует итоговую текстовую расшифровку и передает ее обратно в систему.

Высокая точность распознавания достигается за счет предварительной подготовки модели, включающей обучение на специализированных наборах данных с аннотированными жестами. Алгоритмы модели адаптированы для работы с реальными пользовательскими записями, что позволяет обрабатывать жестовую речь в различных условиях, таких как изменяющееся освещение, различное качество видео и индивидуальные особенности исполнения жестов.

2.5 Блок обучения нейронной сети

Обучение модели заключается в обработке и анализе выходных данных графов MediaPipe, представляющих собой координаты ключевых точек рук и лица на каждом кадре видео. Эти данные формируют основу для

распознавания жестов, позволяя системе анализировать движения во времени.

Сеть обучается на размеченном корпусе данных, содержащем пары входных данных и соответствующих им целевых значений. В данном случае входными данными являются координаты ключевых точек, выделенные графами MediaPipe, а целевыми значениями – текстовые метки, описывающие жесты. На этапе обучения применяется алгоритм оптимизации, который корректирует веса нейронов в каждом слое сети с целью минимизации разницы между предсказанными результатами и истинными значениями, что позволяет снизить ошибку предсказания.

Для корректной работы модели используются функции активации, которые играют ключевую роль в определении, какие нейроны будут активированы в процессе вычислений. Функция активации обеспечивает нелинейность модели, что позволяет сети эффективно моделировать сложные зависимости в данных.

Так, функция Rectified Linear Unit ускоряет обучение за счет своей простоты и способности эффективно пропускать положительные значения, что уменьшает вычислительную сложность и помогает избежать проблемы затухающего градиента.

На выходном слое сети применяется функция SoftMax, которая преобразует выходные значения нейронов в вероятностное распределение по классам. Это позволяет интерпретировать результаты работы модели как вероятности принадлежности к определенным классам, что особенно важно при решении задач классификации распознанных жестов. После завершения обучения модель сохраняется в оптимизированном формате для дальнейшего использования в системе распознавания жестов.

2.6 Блок тестирования

На этапе тестирования модели необходимо, чтобы были полностью подготовлены блоки входных данных, предобработки и сама модель. Тестирование представляет собой процесс измерения производительности модели на новых, ранее не использованных данных. Этот этап позволяет оценить, насколько эффективно модель обобщает полученные знания, а также выявить возможные проблемы, такие как переобучение.

Тестирование в первую очередь проводится для оценки производительности. С помощью таких метрик, как точность (precision), полнота (recall), F1-мера и IoU (Intersection over Union), можно объективно оценить эффективность работы модели.

Также в ходе тестирования выявляется переобучение. Если модель демонстрирует хорошие результаты на обучающих и валидационных данных, но показывает низкие показатели на тестовых данных, это может свидетельствовать о переобучении.

Тестирование позволяет проводить сравнение различных вариантов модели, что помогает выбрать оптимальное решение для поставленной задачи.

Такой структурированный подход к тестированию позволяет не только

оценить качество работы модели, но и выявить слабые места, требующие доработки, что в итоге способствует повышению ее эффективности и надежности.

2.7 Блок подсчета потерь

Блок подсчета потерь отвечает за вычисление функции потерь – математического выражения, которое измеряет разницу между предсказанными значениями модели и истинными метками. Этот блок является ключевым для корректировки весов нейронной сети, так как минимизация потерь приводит к улучшению точности распознавания.

В данный блок могут быть интегрированы следующие функции:

1. Среднеквадратичная ошибка (от англ. Mean Squared Error, MSE). Эта функция вычисляет среднее значение квадратов разностей между предсказанными значениями и истинными метками. За счет возведения ошибки в квадрат большие расхождения оказываются сильнее наказаны, что способствует более точной настройке весов.

2. Средняя абсолютная ошибка (от англ. Mean Absolute Error, MAE). MAE измеряет среднее абсолютное отклонение предсказанных значений от истинных. Эта функция менее чувствительна к выбросам, так как не возводит разницу в квадрат, что может быть полезно, если данные содержат экстремальные значения.

3. Бинарная кросс-энтропия (от англ. Binary Cross-Entropy). Применяется для задач, где целевая переменная принимает два значения, например, 0 или 1. Эта функция измеряет разницу между предсказанными вероятностями и истинными бинарными метками, резко наказывая модель за уверенные, но неверные предсказания.

4. Категориальная кросс-энтропия (от англ. Categorical Cross-Entropy, CCE). Используется в задачах многоклассовой классификации, где целевая переменная может принимать более двух значений. Здесь истинные метки обычно представляются в виде вектора one-hot, и функция сравнивает распределение вероятностей, предсказанное моделью, с истинным распределением.

Выбор конкретной функции потерь зависит от специфики задачи: для регрессии используются функции, оценивающие разницу между числовыми значениями, а для классификации – функции, измеряющие расхождение между распределениями вероятностей. Эффективная реализация подсчета потерь обеспечивает надежное обновление весов модели, что является залогом успешного обучения и качественного распознавания входных данных.

2.8 Блок набора данных

Блок набора данных является основой системы распознавания жестовой речи. Он включает тщательно собранный и размеченный корпус видеоматериалов, где каждая запись снабжена текстовой транскрипцией.

Важнейшим этапом является разметка данных с использованием MediaPipe, который автоматически извлекает ключевые точки рук и лица, формируя базу признаков для обучения модели.

Данные собираются из различных источников, чтобы охватить разнообразные условия съемки и особенности исполнения жестов. После предварительной обработки видео проходит автоматическую разметку через MediaPipe, что обеспечивает высокую точность и согласованность аннотаций. Полученный набор данных делится на обучающую, валидационную и тестовую выборки, позволяя оптимизировать модель и объективно оценивать ее эффективность на новых данных.

2.9 Блок подготовки набора данных

Блок подготовки набора данных отвечает за предварительную обработку и структурирование исходных видеоматериалов, предназначенных для обучения модели распознавания жестов. На данном этапе осуществляется сбор, очистка и стандартизация данных, что позволяет повысить качество разметки и обеспечить корректное извлечение признаков.

Особое внимание уделяется автоматической разметке с использованием MediaPipe. Этот инструмент позволяет точно извлекать координаты ключевых точек рук и лица из видеозаписей, что упрощает процесс формирования аннотаций и снижает вероятность ошибок, связанных с ручной разметкой. Полученные данные нормализуются и приводятся к единому формату для дальнейшей обработки.

Дополнительно реализуются методы аугментации данных, такие как изменение масштаба, поворота и яркости, что способствует увеличению разнообразия обучающего набора. Итоговый набор данных разделяется на обучающую, валидационную и тестовую выборки, что обеспечивает объективную оценку работы модели и помогает избежать переобучения.

3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

3.1 Архитектура программного модуля

Программный модуль телеграм-бота с поддержкой языка жестов представлен следующими основными функциональными блоками:

- блок модели нейронной сети;
- блок обучения нейронной сети;
- блок подготовки набора данных;
- блок данных;
- блок взаимодействия с пользователем;
- блок подсчета потерь;
- блок тестирования;
- блок интеграции с телеграм-ботом.

Описание архитектуры каждого из блоков представлено ниже.

3.2 Блок модели нейронной сети

Нейронная сеть, предназначенная для распознавания языка жестов, реализует концепцию «последовательность-в-последовательность» и состоит из двух ключевых компонентов: энкодера и декодера. Каждый компонент включает в себя специализированные слои и подмодули, обеспечивающие извлечение пространственно-временных признаков из видеопоследовательностей, и преобразование их в текстовую расшифровку.

3.1.1 Энкодер

Энкодер предназначен для обработки входной последовательности графов, где каждый кадр видео представлен в виде набора ключевых точек руки с координатами x , y и z . Он состоит из двух основных частей:

- класс `GCNLayer`;
- класс `Encoder`.

Класс `GCNLayer`, или графовый сверточный слой выполняет агрегацию признаков ключевых точек с учётом их пространственных взаимосвязей, используя матрицу смежности, описывающую топологию руки. В данном классе реализуются следующие ключевые методы:

1. Метод `__init__(self, in_features, out_features)` – инициализирует линейное преобразование, задавая размеры входных и выходных признаков. Принимаемые параметры:

- `in_features`: количество входных признаков, например, 3 для координат x , y , z ;
- `out_features`: количество выходных признаков, или же размер скрытого слоя.

2. Метод `forward(self, x, adj)` – перемножает тензор с данными о ключевых точках и матрицу смежности, и пропускает результат через

линейное преобразование с функцией активации ReLU для усиления нелинейных зависимостей. Принимаемые параметры:

- `x`: тензор признаков узлов;
- `adj`: матрица смежности, описывающая связи между узлами.

Класс `Encoder` отвечает за обработку временной динамики последовательности кадров. Данный модуль обрабатывает каждый кадр видеопоследовательности: сначала для каждого кадра применяется `GCNLayer`, затем выполняется глобальное усреднение по всем ключевым точкам, чтобы получить компактное представление кадра, после чего последовательность этих векторов обрабатывается LSTM для учета временной зависимости жестов. В данном классе реализуются следующие ключевые методы:

1. Метод `__init__(self, node_feature_dim, hidden_dim, num_nodes)` задаёт ключевые параметры и инициализирует `GCNLayer` и LSTM. Принимаемые параметры:

- `node_feature_dim`: размерность признаков узлов;
- `hidden_dim`: размер скрытого состояния;
- `num_nodes`: количество ключевых точек.

2. Метод `forward(self, x_seq, adj)` – проходит по каждому кадру входной последовательности, применяя `GCNLayer` для извлечения локальных признаков, затем выполняет усреднение по всем точкам и передаёт полученную последовательность в LSTM для формирования скрытых представлений, которые затем используются декодером. Принимаемые параметры:

- `x_seq`: тензор последовательности кадров;
- `adj`: матрица смежности.

3.1.2 Декодер

Декодер отвечает за преобразование скрытых представлений, полученных от энкодера, в последовательность токенов, которая интерпретируется как текстовая расшифровка жестов. Он включает следующие ключевые компоненты:

- класс `Decoder`;
- класс `Seq2Seq`;
- класс `SignLanguageModel`.

Класс `Decoder` отвечает за генерацию текстовой последовательности. Этот модуль сначала преобразует входную последовательность токенов в числовые векторы, называемые эмбедингами, с помощью одноименного слоя, затем обрабатывает их через LSTM, а в конце применяет линейное преобразование для получения распределения вероятностей по словарю. В данном классе реализуются следующие ключевые методы:

1. Метод `__init__(self, hidden_dim, vocab_size, embedding_dim)` – настраивает слой эмбединга (для представления токенов в виде векторов),

LSTM для последовательной обработки и линейный слой, выводящий логиты по каждому элементу словаря. Принимаемые параметры:

- `node_feature_dim`: размер скрытого состояния;
- `hidden_dim`: размер словаря токенов;
- `num_nodes`: размер эмбедингов токенов.

2. Метод `forward(self, target_seq, hidden, cell)` – принимает входной текст в виде последовательности токенов и скрытые состояния, преобразует токены в эмбединги, затем обрабатывает их через LSTM, после чего результат пропускается через линейный слой, который возвращает вероятность выбора каждого токена. Принимаемые параметры:

- `target_seq`: последовательность целевых токенов;
- `hidden`: скрытое состояние от энкодера;
- `cell`: ячейка состояния от энкодера.

Класс `Seq2Seq`. Этот класс объединяет работу энкодера и декодера, позволяя обучать модель с использованием техники `teacher forcing`. В данном классе реализуются следующие ключевые методы:

1. Метод `__init__(self, encoder, decoder, device)` – принимает готовые экземпляры энкодера и декодера, а также указывает устройство вычислений – CPU или GPU. Принимаемые параметры:

- `encoder`: экземпляр класса `Encoder`;
- `decoder`: экземпляр класса `Decoder`;
- `device`: устройство вычислений.

2. Метод `forward(self, src, adj, trg)` – последовательно пропускает входные данные через энкодер, затем использует скрытые состояния для запуска декодера, который генерирует итоговую последовательность токенов. Принимаемые параметры:

- `src`: входная последовательность кадров;
- `adj`: матрица смежности;
- `trg`: целевая последовательность токенов.

Класс `SignLanguageModel`. Этот класс служит интерфейсом для работы с моделью, обеспечивая удобную загрузку обученных весов, выполнение жадного декодирования и получение окончательного текстового вывода. В данном классе реализуются следующие ключевые методы:

1. Метод `__init__(self, model_path, vocab, node_feature_dim=3, hidden_dim=64, embedding_dim=32, num_nodes=21, device=None)` – инициализирует энкодер, декодер и `Seq2Seq` модели, загружает веса из файла и устанавливает словарь для токенизации. Принимаемые параметры:

- `model_path`: путь к файлу модели;
- `vocab`: словарь токенов;
- `node_feature_dim`: размерность признаков узлов;
- `hidden_dim`: размер скрытого состояния;
- `embedding_dim`: размер эмбедингов;
- `num_nodes`: количество ключевых точек;
- `device`: устройство.

2. Метод `load_model(self)` – отвечает за загрузку сохранённого состояния модели из указанного файла, проверяя наличие файла и устанавливая веса. Принимаемых параметров, кроме экземпляра класса, нет.

3. Метод `greedy_decode(self, src, adj, max_length=20)` – реализует алгоритм жадного декодирования, при котором генерируется последовательность токенов до появления специального токена конца последовательности или до достижения максимальной длины. Принимаемые параметры:

- `src`: входная последовательность кадров;
- `adj`: матрица смежности;
- `max_length`: максимальная длина последовательности.

4. Метод `predict(self, video_array, adj, max_length=20)` – принимает массив ключевых точек, добавляет размерность батча, выполняет инференс через модель и возвращает текстовую расшифровку жестов, формируя её из сгенерированных токенов по обратному отображению словаря. Принимаемые параметры:

- `video_array`: массив ключевых точек;
- `adj`: матрица смежности;
- `max_length`: максимальная длина.

3.3 Блок подготовки набора данных

Блок подготовки данных играет ключевую роль в получении и предварительной обработке информации из видео. Классов в данном блоке не представлено, однако есть функции, выполняющие определенную роль. Такие ключевые функции описаны ниже:

1. Функция `extract_keypoints_from_video` использует MediaPipe для обработки видео кадр за кадром. Для каждого кадра определяется, обнаружена ли рука, и если да, то извлекаются 21 ключевая точка, представленные координатами x, y, z . В случае отсутствия обнаружения используется нулевой вектор. Принимаемые параметры:

- `video_path`: путь к видео;
- `target_seq_len`: целевая длина;
- `num_keypoints`: количество ключевых точек.

2. Функция `process_mediapipe_data`. Для корректной работы модели полученная последовательность кадров либо дополняется нулевыми значениями, если её длина меньше заданного значения, либо усечается до требуемой длины, что гарантирует единообразие входных данных. Принимаемые параметры:

- `json_file`: путь к JSON-файлу;
- `target_seq_len`: целевая длина последовательности;
- `num_keypoints`: количество ключевых точек.

3. Функция `create_hand_adjacency_matrix` формирует матрицу смежности, описывающую пространственные связи между ключевыми

точками руки. Эта матрица затем используется в графовом сверточном слое энкодера для эффективного агрегирования пространственных признаков. Принимаемые параметры:

- `num_keypoints`: количество ключевых точек.

3.4 Блок обучения нейронной сети

Блок обучения позволяет перейти от сырого набора данных к текстовым меткам, путем вычисления закономерностей и адаптации параметров нейросети. Блок включает следующий класс:

- класс `MediaPipeSignLanguageDataset`.

Класс `MediaPipeSignLanguageDataset` играет ключевую роль в процессе подготовки данных для обучения модели распознавания жестового языка. Его основная задача – загрузить данные из JSON-файлов, содержащих координаты ключевых точек для каждого кадра видео, и соответствующие текстовые метки. Эти данные затем преобразуются в формат, который может быть использован для обучения нейронной сети, работающей с последовательностями и графами. В данном классе реализуются следующие ключевые методы:

1. Метод `__init__(self, json_paths, labels, target_seq_len, num_keypoints, vocab)`. Этот метод инициализирует экземпляр набора данных, подготавливая все необходимые структуры данных для дальнейшей работы. Он сохраняет переданные параметры, такие как пути к файлам, метки и настройки обработки, а также создаёт матрицу смежности — структуру, которая описывает связи между ключевыми точками руки, например, между пальцами или суставами. Принимаемые параметры:

- `json_paths`: список путей к JSON-файлам;
- `labels`: список меток;
- `target_seq_len`: длина последовательности кадров;
- `num_keypoints`: количество ключевых точек;
- `vocab`: словарь токенов.

3.5 Блок интеграции с Telegram-ботом

Для обеспечения интерактивного взаимодействия с пользователем разработан модуль, интегрирующий модель распознавания жестов с Telegram-ботом. Основные функции этого блока таковы:

Приём видео от пользователя. Бот, реализованный с использованием библиотеки `python-telegram-bot`, обрабатывает входящие видео-сообщения. При получении сообщения бот извлекает файл видео, сохраняет его локально и уведомляет пользователя о начале обработки.

Извлечение признаков и инференс. После получения видео вызывается функция из модуля `dataprocessor`, которая извлекает ключевые точки из видео. Затем, с помощью метода `predict` класса `SignLanguageModel`, производится

инференс модели, который генерирует текстовую расшифровку жестов.

Отправка результата пользователю. Полученный текст отправляется обратно пользователю в виде сообщения, обеспечивая оперативную обратную связь.

Модуль Telegram-бота организован в файле `telegram_bot`, где подробно прописаны обработчики команд и видео-сообщений, а также настройки подключения к Telegram API.

3.6 Зависимости и организация проекта

Проект структурирован модульно для обеспечения простоты поддержки и расширения функциональности:

Модуль `model` содержит классы, реализующие архитектуру нейронной сети – `GCNLayer`, `Encoder`, `Decoder`, `Seq2Seq` – а также обёртку `SignLanguageModel`, предоставляющую удобный интерфейс для инференса

Модуль `dataprocessor` включает функции для обработки входных данных: извлечение ключевых точек из видео и построение матрицы смежности

Модуль `telegram_bot` отвечает за интеграцию модели с Telegram-ботом, организуя получение видео, запуск инференса и отправку результатов пользователю

Эта организация позволяет независимо тестировать и дорабатывать каждый компонент, легко интегрировать новые методы обработки или изменять архитектуру модели, а также обеспечивает масштабируемость и повторное использование кода в других приложениях.

7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И РЕАЛИЗАЦИИ НА РЫНКЕ ТЕЛЕГРАМ-БОТА С ПОДДЕРЖКОЙ ЯЗЫКА ГЛУХОНЕМЫХ

7.1 Характеристика программного средства, разрабатываемого для реализации на рынке

Современный мир активно стремится к выравниванию возможностей среди различных слоев населения планеты. Такие тенденции коснулись в том числе и людей с ограничениями в слухе или речи, что непосредственно повлияло на спрос продуктов, стирающих барьеры в общении между глухонемыми и слышащими людьми. Данное программное средство разрабатывается с целью упрощения коммуникации между такими группами людей, и их друзьями и родственниками, не имеющих подобных особенностей.

Область применения:

- повседневная жизнь;
- коммерческая и научная деятельность.

Решаемые задачи:

- улучшение коммуникации между социальными группами;
- упрощение коммуникации с людьми с особенностями;
- интеграция в современный популярный мессенджер.

Функции:

- преобразование видео и видео-сообщений в текст.

Целевая аудитория:

- люди с нарушением слуха и речевого аппарата;
- друзья и родственники людей с нарушением слуха и речи.

7.2 Расчет инвестиций в разработку программного средства для его реализации на рынке

Разработка телеграм-бота с поддержкой языка глухонемых включает несколько этапов, каждый из которых требует определенных финансовых вложений. Основные затраты на этапе разработки связаны с оплатой труда специалистов, использованием программного обеспечения, вычислительных ресурсов и других вспомогательных инструментов. Расчет затрат на заработную плату разработчиков представлен в таблице 7.1.

Основная зарплата определяется по формуле 7.1:

$$Z_o = K_{\text{пр}} \cdot \sum_{i=1}^n Z_{\text{чи}} \cdot t_i, \quad (7.1)$$

где $K_{\text{пр}}$ – коэффициент премий и иных стимулирующих выплат;

n – категории исполнителей, занятых разработкой продукта;

$Z_{\text{ч}i}$ – часовая заработная плата исполнителя i -й категории, р.;

t_i – трудоемкость работ, выполняемых исполнителем i -й категории, ч.

Для реализации проекта необходим один инженер-программист, один инженер по машинному обучению и тестировщик. В задачи инженера по машинному обучению входит разработка алгоритма и обучение модели нейронной сети. В задачи инженера-программиста входит реализация серверной и клиентской частей для взаимодействия нейронной сети с пользователем. Инженер-тестировщик занимается выявлением ошибок системы, тестированием требований, а также отвечает за приемку качества поставляемого продукта.

Месячная заработанная плата инженеров основана на медианных показателях для специалистов в области информационных технологий [5-7].

Часовая заработная плата каждого исполнителя определяется путём деления его месячной заработной платы (оклад плюс надбавки) на количество рабочих часов в месяце (расчётная норма рабочего времени на 2025 г. для 40-часовой рабочей недели составляет 167 часов по данным Министерства труда и социальной защиты населения на момент проведения расчётов).

Таблица 7.1 – Расчет затрат на основную заработную плату

Категория исполнителя	Месячный оклад, р.	Часовой оклад, р.	Трудоёмкость работ, ч.	Итого, р.
Инженер-программист	3400	20,40	140	2856
Инженер по машинному обучению	3600	21,60	120	2592
Инженер-тестировщик	2200	13,10	80	1048
Итого				64960
Премия и иные стимулирующие выплаты, 10%				649,60
Всего затраты на основную заработную плату разработчиков				7145,60

Дополнительная заработная плата определяется по формуле 7.2:

$$Z_{\text{д}} = \frac{Z_{\text{о}} \cdot H_{\text{д}}}{100}, \quad (7.2)$$

где $H_{\text{д}}$ – норматив дополнительной зарплаты, 15%.

Отчисления в фонд социальной защиты населения и обязательное страхование БелГосстрах ($Z_{\text{сз}}$) определяется в соответствии с действующим законодательством по формуле 7.3:

$$P_{\text{соц}} = \frac{(Z_{\text{о}} + Z_{\text{д}}) \cdot H_{\text{соц}}}{100}, \quad (7.3)$$

где $H_{\text{соц}}$ – норматив отчислений в ФСЗН и Белгосстрах (в соответствии с действующим законодательством по состоянию на март 2025 г. – 35%).

Затраты на прочие расходы рассчитываются по формуле 7.4:

$$P_{\text{пр}} = \frac{3_o \cdot H_{\text{пр}}}{100}, \quad (7.4)$$

где $H_{\text{пр}}$ – норматив прочих расходов, 30%.

Расходы на реализацию рассчитываются по формуле 7.5:

$$P_p = \frac{3_o \cdot H_p}{100}, \quad (7.5)$$

где H_p – норматив расходов на реализацию, 3%.

Чтобы найти общую сумму затрат на разработку и реализацию нужно воспользоваться формулой 7.6:

$$3_p = 3_o + 3_d + P_{\text{соц}} + P_{\text{пр}} + P_p, \quad (7.6)$$

Расчёт затрат на разработку программного средства представлен в таблице 7.2.

Таблица 7.2 – Затраты на разработку

Название статьи затрат	Формула/таблица для расчёта	Сумма, р.
1 Основная заработная плата разработчиков 3_o	Таблица 7.1	7145,60
2 Дополнительная заработная плата разработчиков	$3_d = \frac{7145,60 \cdot 15}{100}$	1071,84
3 Отчисление на социальные нужды	$P_{\text{соц}} = \frac{(7145,60 + 1071,84) \cdot 35}{100}$	3572,80
4 Прочие расходы	$P_{\text{пр}} = \frac{7145,60 \cdot 30}{100}$	2143,68
5 Расходы на реализацию	$P_p = \frac{7145,60 \cdot 3}{100}$	214,37
6 Общая сумма затрат на разработку и реализацию	$3_p = 7145,60 + 1071,84 + 3572,80 + 2143,68 + 214,37$	14148,29

7.3 Расчет экономического эффекта от реализации программного средства на рынке

В данном разделе проводится расчет экономического эффекта от реализации программного средства, что позволит оценить его коммерческую привлекательность и потенциальную прибыльность.

Для оценки цены программного средства необходимо произвести анализ продуктов-аналогов. В таблице 7.3 показаны продукты-аналоги и их цена за использование [8-11].

Таблица 7.3 – Цена использования продуктов-аналогов

Название продукта	Цена за использование
Vidby	от 35р./мес.(подписочная модель)
HeyGen	от 85р./мес.(подписочная модель)
Wavel AI	от 2,50р./мин. видео
Descript	от 35р./мес.(подписочная модель)

Учитывая, что разрабатываемое программное средство распространяется через мессенджер Telegram, наиболее оптимальной моделью монетизации является подписочная модель.

На основе анализа конкурентов, предлагается установить цену 10р. ($C_{отп}$) за месяц подписки с доступом к неограниченному использованию функционала приложения.

Для повышения привлекательности сервиса можно рассмотреть введение:

- ежедневного бесплатного лимита в пять видео или видео-сообщений для одного пользователя;
- единовременная покупка неограниченного доступа к сервису за 35р.

Такой подход увеличит вероятность успешного закрытия сделки с потенциальным покупателем, так как он сможет опробовать товар перед покупкой. А также увеличит прибыль от реализации продукта в моменте, за счет продажи пожизненного полного доступа, что очень важно на старте продукта.

Прирост чистой прибыли, полученной от реализации программного средства на рынке, можно рассчитать по формуле 7.7:

$$\Delta\Pi^p = (C_{отп} \cdot N - \text{НДС}) \cdot P_{пр} \cdot \left(1 - \frac{H_{п}}{100}\right), \quad (7.7)$$

где $C_{отп}$ – отпускная цена за месяц пользования приложением, р.;

N – количество месяцев оплаченных пользователями за год, шт.;

НДС – сумма налога на добавленную стоимость, р.;

$P_{пр}$ – рентабельность продаж, 20%;

$H_{п}$ – ставка налога на прибыль, 25%.

Налог на добавленную стоимость определяется по формуле 7.8:

$$\text{НДС} = \frac{C_{отп} \cdot N \cdot H_{д.с.}}{100 + H_{д.с.}}, \quad (7.8)$$

где $H_{д.с.}$ – ставка налога на добавленную стоимость, 20%.

$$\text{НДС} = \frac{10 \cdot 600 \cdot 20}{100 + 20} = 1000\text{р.}$$

$$\Delta\Pi_{\text{ч}}^{\text{р}} = (10 \cdot 600 - 1000) \cdot 20 \cdot \left(1 - \frac{25}{100}\right) = 75000\text{р.}$$

7.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке

Оценка экономической эффективности разработки и реализации телеграмм-бота с поддержкой языка глухонемых, основывается на сравнении инвестиций в его разработку и годового прироста чистой прибыли, полученной от его реализации на рынке. В данном разделе представлены расчеты и анализ экономической эффективности проекта.

Поскольку сумма инвестиций на разработку программного средства меньше суммы годового экономического эффекта, оценка экономической эффективности осуществляется с помощью расчета рентабельности инвестиций (Return on Investment, ROI). ROI определяется по формуле 7.9:

$$\text{ROI} = \frac{\Delta\Pi_{\text{ч}}^{\text{р}} - \text{З}_{\text{р}}}{\text{З}_{\text{р}}}, \quad (7.9)$$

где $\Delta\Pi_{\text{ч}}^{\text{р}}$ – прирост чистой прибыли, полученной от реализации программного средства на рынке, р.;

$\text{З}_{\text{р}}$ – затраты на разработку и реализацию программного средства, р.

$$\text{ROI} = \frac{75000 - 14148,29}{14148,29} = 430\%$$

Такой уровень рентабельности подтверждает целесообразность реализации проекта и его потенциал для дальнейшего развития.

7.5 Анализ полученных результатов

На основе проведенных расчетов и анализа показателей экономической эффективности можно сделать вывод, что разработка и реализация телеграмм-бота с поддержкой языка глухонемых является экономически обоснованным и перспективным проектом. Значение рентабельности инвестиций (ROI), равное 430%, свидетельствует о высокой эффективности вложений, что превышает доходность большинства альтернативных вариантов инвестирования, таких как банковские депозиты или фондовый рынок.

Эффективность разработки и реализации:

1. Окупаемость проекта: инвестиции в разработку программного средства окупаются в течение первого года реализации, что подтверждается

положительным значением ROI.

2. Прибыльность: годовой прирост чистой прибыли составляет 75000р., что демонстрирует коммерческую привлекательность продукта.

3. Конкурентоспособность: установленная цена ежемесячной подписки в 10р. является конкурентоспособной.

Несмотря на высокие показатели эффективности, реализация программного продукта на массовом рынке сопряжена с определенными рисками, которые необходимо учитывать:

1. Технологические риски: Быстрое развитие технологий может привести к появлению более совершенных решений, что снизит привлекательность продукта.

2. Операционные риски: Зависимость от вычислительных ресурсов и возможные сбои в работе сервиса могут негативно сказаться на репутации продукта.

3. Маркетинговые риски: Недостаточное продвижение продукта может привести к низкому уровню продаж, несмотря на его технические преимущества.

ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта была изучена область нейросетевого анализа изображений и взаимодействия с Telegram Bot API. Проведён анализ существующих решений для обработки видеопотока и распознавания жестов, а также рассмотрены методы оптимизации нейросетей для работы в реальном времени.

В процессе практики выполнены следующие ключевые этапы:

- исследование и выбор архитектуры нейронной сети для распознавания жестов;
- подготовка и предобработка данных с использованием ключевых точек MediaPipe;
- интеграция модели в Telegram-бота для обработки видеофайлов и генерации текстовых ответов;
- реализация механизма загрузки и декодирования видео с помощью Telegram Bot API.

Были подготовлены следующие графические материалы:

- вводный плакат;
- схема структурная;
- диаграмма классов пользовательских интерфейсов;
- диаграмма последовательности пользовательского интерфейса.

Также был проведён анализ рынка, рассчитана примерная стоимость продукта и выполнено технико-экономическое обоснование его реализации на массовом рынке.

Использование Telegram-бота в качестве интерфейса взаимодействия обеспечивает удобный доступ к функционалу без необходимости установки дополнительного программного обеспечения. Гибкость архитектуры позволяет адаптировать модель для новых сценариев использования, а также расширять её возможности с минимальными изменениями в коде.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Траск, Э. Грокаем глубокое обучение / Э. Траск. – СПб : Питер, 2022. – 352 с.
- [2] Пример задач, решаемых с помощью MediaPipe [Электронный ресурс] – Электронные данные. – <https://ai.google.dev/edge/mediapipe/solutions/examples>
- [3] MediaPipe [Электронный ресурс] – Электронные данные. – Режим доступа: <https://ai.google.dev/edge/mediapipe/solutions/guide>
- [4] Telegram Bot API [Электронный ресурс] – Электронные данные. – Режим доступа: <https://core.telegram.org/bots/api>
- [5] Зарплаты инженеров машинного обучения [Электронный ресурс] – Электронные данные. – Режим доступа: <https://geeklink.io/salary-stat/machine-learning-engineer/>
- [6] Зарплаты Python-разработчиков [Электронный ресурс] – Электронные данные. – Режим доступа: <https://geeklink.io/salary-stat/python-razrabotchik/>
- [7] Зарплаты тестировщиков ПО [Электронный ресурс] – Электронные данные. – Режим доступа: <https://geeklink.io/salary-stat/testirovshhik-po/>
- [8] Vidby – Сервисный план [Электронный ресурс] – Электронные данные. – Режим доступа: <https://vidby.com/ru/prices>
- [9] HeyGen – Сервисный план [Электронный ресурс] – Электронные данные. – Режим доступа: <https://www.heygen.com/pricing>
- [10] Wavel AI – Сервисный план [Электронный ресурс] – Электронные данные. – Режим доступа: <https://ru.wavel.ai/solutions/ai-video-translator>
- [11] Descript – Сервисный план [Электронный ресурс] – Электронные данные. – Режим доступа: <https://www.descript.com/pricing>
- Стандарт предприятия. Дипломные проекты (работы). Общие требования. СТП 01-2024 [Электронный ресурс] – Электронные данные. – Режим доступа: https://www.bsuir.by/m/12_100229_1_185678.pdf

ПРИЛОЖЕНИЕ А
(обязательное)

Вводный плакат. Плакат

ПРИЛОЖЕНИЕ Б
(обязательное)

Схема структурная

ПРИЛОЖЕНИЕ В

(обязательное)

**Диаграмма классов. Пользовательский интерфейс. Диаграмма
последовательности**